

TITLE

Pedram

Giovanni Galea Curmi



April 15, 2026

1 Introduction

GPU kernel performance depends not only on what computation is being performed, but also on how that computation is mapped to the hardware. Choices such as tile sizes, warp counts, and pipeline stages can make a large difference in runtime, even when two kernels compute the same output.

Triton makes GPU kernel programming easier than raw CUDA by giving programmers a higher-level language for writing efficient kernels [1, 2]. However, Triton does not remove the need to choose good kernel configurations. A kernel such as GEMM or Layer-Norm can be correct under many settings, but those settings can vary widely in performance. In practice, the usual solution is to benchmark several candidate configurations and keep the best one, either manually or through built-in autotuning tools such as `triton.autotune` [3]. This works, but it can be expensive when the configuration space is large or when tuning must be repeated across many workloads.

This project studies whether that tuning process can be made more efficient. Instead of benchmarking every valid configuration, we ask whether cheap signals such as compile-time resource information and simple occupancy estimates can help rule out poor candidates early, and whether a small amount of hardware profiling can further improve the search. Our goal is not to redesign Triton, replace vendor libraries, or generate new kernels from scratch. Instead, we focus on a narrower question: can a bottleneck-aware selector choose better Triton configurations than defaults or naive search under the same measurement budget?

This places our work in a middle ground within the broader optimization literature. It is narrower than Halide and TVM-style autoscheduling systems, which search over much richer schedule spaces [4, 5, 6, 7, 8, 9]. It is also narrower than recent agentic CUDA systems such as CUDA Agent, which optimize or generate full CUDA kernels rather than selecting configurations for a fixed Triton kernel [10].

KernelTuner keeps the Triton kernel fixed and asks a simpler but still meaningful question: for a bounded Triton configuration space, can a bottleneck-aware selector outperform defaults or naive search under the same budget?

To study this question, we build a framework that generates candidate configurations, collects cheap signals, benchmarks candidate kernels, optionally profiles a limited calibration subset, and compares a bottleneck-aware selector against matched-budget baselines. The system also stores structured artifacts for later analysis, making it possible not only to measure whether the selector helps, but also to explain when it helps, when it fails, and why.

Our main contribution is therefore not a new compiler and not a new code-generation system. Instead, it is a practical and reproducible framework for bottleneck-aware Triton configuration selection, together with an empirical study of whether lightweight signals and limited profiling are sufficient to guide tuning effectively.

2 Related Work

Our work is most closely connected to four lines of prior research: autoscheduling systems, tensor compiler autotuning, external kernel autotuners, and recent agentic GPU optimization methods.

The first line of work comes from Halide. Halide introduced the idea of separating the algorithm from the schedule, making optimization decisions explicit rather than mixing them with correctness logic [4]. This led naturally to the problem of automatic scheduling. Later work showed that schedule search could be guided automatically, including with learned methods that explore large schedule spaces while still finding strong implementations [5, 6]. In these systems, the optimization target is a rich schedule language with many possible transformations over locality, recomputation, and parallel execution.

A second line of work comes from tensor compil-

ers such as TVM. AutoTVM introduced learning-based cost models for optimizing tensor programs [7]. Ansor extended this by automating schedule generation more broadly through sketch-based search and cost-model guidance [8]. More recent work such as MetaSchedule and TensorIR continued this direction by using richer tensor-IR representations and more expressive optimization strategies [9, 11]. These systems are closely related to our project because they also study search over implementation choices, but they operate at a much broader compiler level than the bounded Triton configuration spaces we consider here.

Triton sits at a narrower and more practical layer. It provides a GPU kernel language in which performance still depends heavily on a compact set of meta-parameters such as tile sizes, `num_warps`, and `num_stages` [1, 2]. Triton already includes built-in tuning support through `triton.autotune` and `triton.heuristics` [3, 12], and the official tutorials show this pattern clearly for kernels such as matrix multiplication [13, 14]. However, Triton’s native autotuning is still mostly benchmark-driven: candidate configurations are enumerated and measured directly. This leaves open the question of whether tuning can be guided more efficiently by cheap signals and limited profiling.

The systems closest in spirit to our work are external autotuners such as Kernel Tuner, CLTune, and the Kernel Tuning Toolkit [15, 16, 17]. These frameworks treat kernels as parameterized programs and provide infrastructure for generating candidate settings, benchmarking them, validating correctness, and comparing search strategies. Our work follows this general direction, but focuses specifically on Triton kernels and on a bottleneck-aware selection problem. Rather than asking a compiler to synthesize schedules, we ask whether a selector can use compile-side signals and limited profile data to guide search over a bounded configuration space.

Recent work on agentic GPU optimization takes a broader approach. CUDA Agent treats high-performance CUDA generation as a reinforcement learning problem over full CUDA kernels [10]. Related efforts such as KernelBench also reflect growing interest in automatic low-level kernel generation and evaluation [18]. These systems are useful context, but they target a different problem. Our work does not generate new kernels. Instead, it keeps the Triton implementation fixed and studies whether configuration search can be made cheaper, fairer, and more interpretable.

Finally, practical evidence from the Triton and PyTorch ecosystem suggests that autotuning remains

an open engineering problem. Official documentation presents autotuning as a standard workflow [19, 3, 13], while issue trackers show that real systems still face correctness edge cases, dynamic-shape interactions, and autotune fragility [20, 21, 22, 23]. Tools such as `triton-dejavu` explicitly aim to reduce autotuning overhead and improve reuse across repeated runs [24]. These sources are not the core scientific basis for our work, but they help motivate why lightweight, reproducible, and budget-aware Triton tuning is still worth studying.

3 Hypothesis

Can cheap compile-time signals and limited profiling guide Triton kernel configuration search better than default settings or naive search under the same budget?

Our hypothesis is that many poor Triton configurations can be identified without benchmarking the full search space. Signals such as register count, shared-memory use, and occupancy estimates may be enough to prune clearly weak candidates, while a small amount of profiling may further improve ranking among the remaining configurations. If this is true, then a bottleneck-aware selector should either find faster configurations under the same budget or achieve similar performance with fewer measurements.

4 Experiment

4.1 Methodology

4.2 Results

References

- [1] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019. 1, 2
- [2] OpenAI. Introducing triton, 2021. 1, 2
- [3] Triton. triton.autotune documentation, 2026. 1, 2
- [4] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Decoupling algorithms from schedules for easy optimization of image processing pipelines. *ACM Transactions on Graphics*, 31(4), 2012. 1
- [5] Ravi Teja Mullanpudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahian. Automatically scheduling halide image processing pipelines. In *ACM SIGGRAPH*, 2016. 1
- [6] Andrew Adams, Rial Baghdadi, John Lotspeich, Shoaib Ma, Yaqi Alisafae, Aaron Anderson, Shoaib Kamil, Laurens Van Der Maaten, Julien Killebrew, and Jonathan Ragan-Kelley. Learning to optimize halide with tree search and random programs. In *ACM SIGGRAPH*, 2019. 1
- [7] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems*, 2018. 1, 2
- [8] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *USENIX OSDI*, 2020. 1, 2
- [9] Tianqi Chen, Minmin Sun, Haichen Shen, and et al. Tensor program optimization with probabilistic programs. *arXiv preprint arXiv:2205.13603*, 2022. 1, 2
- [10] et al. Dai. Cuda agent: Large-scale agentic rl for high-performance cuda kernel generation. *arXiv preprint arXiv:2602.24286*, 2026. 1, 2
- [11] Lianmin Zheng, Chengfan Jia, Minmin Sun, and et al. Tensorir: An abstraction for automatic tensorized program optimization. *arXiv preprint arXiv:2207.04296*, 2022. 2
- [12] Triton. triton.heuristics documentation, 2026. 2
- [13] Triton. Matrix multiplication tutorial, 2026. 2
- [14] Triton. Persistent matmul tutorial, 2026. 2
- [15] Ben van Werkhoven. Kernel tuner: A search-optimizing gpu code auto-tuner. *Future Generation Computer Systems*, 2019. 2
- [16] Cedric Nugteren and Henk Corporaal. Cltune: A generic auto-tuner for opencl kernels. In *MCSoc*, 2015. 2
- [17] Tomáš Petrov and et al. Kernel tuning toolkit. *SoftwareX*, 2023. 2
- [18] Scaling Intelligence. Kernelbench, 2025. 2
- [19] PyTorch. torch.compile documentation, 2026. 2
- [20] PyTorch. Issue #120802: Custom triton autotune mutation / correctness discussion, 2024. 2
- [21] PyTorch. Issue #141823: Triton autotune / codegen edge case, 2024. 2
- [22] PyTorch. Issue #147051: Triton gemm autotune fragility, 2025. 2
- [23] PyTorch. Issue #164124: Dynamic-shape / autotune issue, 2025. 2
- [24] IBM. triton-dejavu, 2025. 2